勉強記録 (2026-04-02)

トピック: [MLA-C01のタスク1.1-1.3のSkillBuilder視聴]

1. 試験での出題ポイント (MLA-C01)

- **タスク 1.1: データを取り込んで保存する**
  - ストレージのアクセスパターン、データ形式・構造、コスト・パフォーマンスに基づいたデータ取
  - スケーラビリティやキャパシティーに関する問題のデバッグとトラブルシューティング。
- **タスク 1.2: データを変換し、特徴量エンジニアリングを実行する**
  - データのクリーンアップ、変換手法。
  - データの探索、可視化、特徴量エンジニアリング、エンコーディング、検証、ラベル付けに役立つ
- **タスク 1.3: データの完全性を確保し、モデリングに向けてデータを準備する**
  - データの分類、匿名化、マスキング手法、および暗号化に役立つAWSサービス。
  - データの完全性と品質の検証、予測におけるバイアスの特定と軽減。

2. 実務でのユースケース

3. 副業案件での需要度と単価感

4. 学んだこと・メモ

- **分野1の全体像**: 機械学習ライフサイクルの最初の2ステップ（データ収集と、品質向上のためのデータ

タスク 1.1「データを取り込んで保存する」の主な学び

- **データ取り込みと処理の主要AWSサービス**
  - **Amazon Athena**: ETL不要でS3上の各データ形式を直接クエリ可能。列指向で保存し圧縮を効か
  - **AWS Lambda**: データの解析、変換、データウェアハウス/DBへのロードタスクを実行。
  - **AWS Glue**: PythonベースのETLフレームワークで、複雑な変換ロジック（クレンジング、集約、正
  - **Amazon EMR**: Apache Spark等を利用してEC2クラスタで分散処理。HDFSを用いたり、パーテ
  - **Amazon Kinesis**: AWS SDKやKCLを使用してストリーミング（リアルタイム）データを処理。
  - **AWS Step Functions**: サーバーレスなワークフローやステートマシンのオーケストレーション。
- **シナリオとアーキテクチャ例**
  - **リアルタイム処理の注意点**: 『リアルタイム』のインサイトが求められる場合、Athenaは不適切。**Service for Apache Flink** と **Lambda** を使って変換・分析するアプローチが正しい。
  - **データ移行と学習の最適化**: オンプレミスDBからS3へは **AWS DMS** を使用し、クエリ高速化のためParquet形式等を利用。S3からSageMakerへの連携では **パイプモード** を採用することで、データ転送を高速化し、学習時の時間・ディスク要件・コストを
  - **特徴量の取り込み**: SageMaker Feature Storeへは、EMR（Sparkコネクタ経由）によるバッチ取り Canvas(Data Wrangler)で特徴量エンジニアリングを行った後エクスポートする構成を取る。
  - **EBSのパフォーマンス最適化**: EC2からEBSボリュームのデータを抽出する際、ネットワークの競
  - **変換パイプラインの運用・コスト削減**: 複数のカスタムソースからデータを取り込む環境において、+AWS Data Pipeline」の構成は運用管理負荷とコストが高い。サーバーレスな **AWS Glue (ETLジョブ)** に移行し、S3へ出力するアーキテクチャに再設計することで、大幅なコ

タスク 1.2「データを変換し、特徴量エンジニアリングを実行する」の主な学び

- **データタイプとエンコーディング**
  - 学習アルゴリズムは基本的に数値を要求するため、カテゴリ特徴量等の適切な処理が必須となる。
  - **カテゴリ変数のエンコーディング**: 予測精度向上のため、数値化（one-hotエンコーディングなど）を行う。
  - 代表的なAWSソリューション:
    - **AWS Glue DataBrew**: レシピアクションを使い、コードを書かずに集計・要約や高度な変換
    - **Amazon SageMaker Canvas (Data Wrangler)**: GUI・ローコードで同様にデータを変換
- **特徴量エンジニアリングと前処理アルゴリズム**
  - 単純な変換だけでなく、モデルのパフォーマンスと学習効率を向上させるため、**特徴量削減アルゴ**PCAなど）を用いる意義を理解する。
- **Amazon SageMaker Canvas (Data Wrangler) の活用と統合**
  - **柔軟なインポート元**: S3、Amazon Redshift、Amazon Athenaからデータを直接インポート。
  - **ノーコード＆カスタムスクリプト**: UI上での可視化・分析・変換に加え、独自のPythonスクリプト
  - **エコシステムとの連携**: 作成したデータフローを **SageMaker Processing**（分散バッチ処理）、**SageMaker Pipelines**（CI/CD）、**SageMaker Feature Store** と連携できる。また構成全体をPythonコードとしてエクスポートし、自前のパイ

タスク 1.3「データの完全性を確保し、モデリングに向けてデータを準備する」の主な学び

- **バイアスの検出とモデルの説明性**
  - モデルの予測精度や公平性を高めるために、数値・テキスト・画像のデータに対して**トレーニング**
  - **Amazon SageMaker Clarify**: モデルの予測理由を説明し（Explainability）、バイアスがトレー
- **基盤モデルの評価**
  - 言語モデル等に対して、基盤モデル評価機能を使用することで、モデルの品質や「責任あるAI」に
- **推論環境での継続的モニタリング**
  - モデルをエンドポイントにデプロイした後も、学習時のベースラインに照らし合わせてバイアスの
- **クラスの不均衡 (Class Imbalance) への対処とデータラベル付け**
  - 高精度なモデル学習には大規模で高品質なラベル付きデータセットが必要であり、不均衡データ
  - **Amazon SageMaker Ground Truth**: 組み込みのラベリングタスクやカスタムワークフローを利
- **データ形式とSageMakerの入力モード (Input Modes)**
  - フレームワークやアルゴリズムによって適切なデータ形式（CSV, JSON, RecordIO, NumPy等）が異なる。
  - **ファイルモード (File Mode)**: データセット全体をS3からEBSボリュームにダウンロード/コピーし
  - **パイプモード (Pipe Mode)**: データをS3から直接ストリーミングするため、ファイルモードより高
  - データの配置場所として、SageMakerは Amazon S3、Amazon EFS、Amazon FSx for Lustre をサポートする。
- **データ品質の測定**
  - **AWS Glue Data Quality**: データ品質定義言語（DQDL）を用いて品質の測定とモニタリングを行う。エントリポイントとして「AWS Glue Data Catalog」と「AWS Glue ETL ジョブ」の2か所を使用する。
- **推論パイプライン (Inference Pipelines)**
  - 生データに対して毎回アプリ側で前処理コードを実行するのは非効率。
  - 学習時に作成したscikit-learnコンテナやSpark ML Servingコンテナの「データ変換」プログラムを
    推論パイプライン を構築できる。

  - これにより、推論パイプライン全体を単一のSageMakerモデルとして扱い、リアルタイム予測と
- **(補足) データクリーンアップと変換機能**
  - **SageMaker Canvas (Data Wrangler)** には、コーディングなしでデータのクリーンアップや特徴

分野2: MLモデルの開発

タスク 2.1「モデリングアプローチを選択する」の主な学び

- **アルゴリズムの種類と深層学習**
  - 分析対象に応じ、古典的機械学習（回帰、SVM、決定木、K-means、ランダムカットフォレスト）から深層学習までを選択する。
  - **CNN (畳み込みニューラルネットワーク)**: 主に画像分類に利用。
  - **RNN (リカレントニューラルネットワーク)**: 順序や時間ベースの入力データ（例: 機械翻訳など）に利用。
- **ニューラルネットワークの課題軽減策**
  - **勾配消失問題**への対応として、ゲート機構により長期の記憶の保持・忘却を判別する **LSTM** が有効。
  - しかしLSTMは構造が複雑で学習に時間がかかるため、必要処理が少なくLSTMよりシンプルなア **GRU (ゲート付き回帰型ユニット)** を用いることで、**トレーニング時間を短縮**できる。
- **モデルの解釈可能性 (Interpretability)**
  - 一般的に、モデルの予測パフォーマンスと解釈可能性（なぜその予測が行われたかの明確さ）の間
  - **本質的分析**: 線形回帰など、モデルの構造自体が単純で、入力変数と予測の関係が自明なものに対
  - **事推的分析 (Post-hoc Analysis)**: ニューラルネットワークなど複雑なモデルに対して適用する。

AWS 機械学習スタックと主要な AI サービス

- **AWS 機械学習スタックの構造**
  - 下層から順に「インフラ/フレームワーク層」、「MLサービス層 (Amazon SageMaker等)」、「AIサービス層」の3層構造。
  - **AIサービス**は、AWSが事前学習済みのモデルをAPIとして提供するもので、機械学習の専門知識が
- **代表的な AI サービスとユースケース**
  - **Amazon Lex**: チャットボットエンジン（コールセンターボットなど）。
  - **Amazon Polly**: 音声をレンダリング（TTS）。業界用語や略語の読み間違いを防ぐには、**カスタ**
  - **Amazon Textract**: 画像から文字を抽出（OCR機能）。手書きやフォーム入力の読み取り。
  - **Amazon Comprehend**: 自然言語処理（NLP）。センチメント（感情）分析やキーフレーズ抽出
  - **Amazon Transcribe**: 音声をテキストに変換（ASR）。ミーティングや通話の文字起こし。
  - **Amazon Translate**: 多言語のテキスト翻訳（ニューラル機械翻訳）。
  - **Amazon Rekognition**: 画像・動画の分析。物体、シーン、顔などの検出を行う。独自の画像で学 **Custom Labels** 機能も持つ。
  - **Amazon Bedrock**: フルマネージドで基盤モデルを活用し、生成AIアプリケーションを構築するた
- **独自データでのトレーニングが必須となる AI サービス**
  - 一部のAIサービスは、自社の過去データ等を用意し、学習（モデルトレーニング）させてから推論
  - **Amazon Forecast**: 過去の時系列データを利用した需要・売上予測。
  - **Amazon Personalize**: 顧客や商品の過去の行動・属性履歴を利用したパーソナライズ・レコメン
  - **Amazon Fraud Detector**: 過去の正常・不正データを用いたオンライン不正行為の検出。

Amazon SageMaker と MLスタックの詳細

- **SageMaker の本質とコンテナの独自運用**
  - MLOpsを実現するための統合ツールセットであり、Python SDK、Jupyter Notebook、ラベリングなどを提供。
  - **コンテナ(Docker)の使われ方**: ECSやEKSのようなオーケストレーションとは異なり、SageMake
  - **コンテナの種類**: TensorFlow等フレームワーク向けの「事前構築済みコンテナ」や、画像分類等に
- **SageMaker の主要機能・サービス群**
  - **Ground Truth**: MLモデル構築の前提となる高品質なデータラベル付け。
  - **ノートブックインスタンス**: Jupyterの実行環境。
  - **ハイパーパラメータチューニングジョブ**: 自動でパラメーターを変えながら複数の学習ジョブを一括
  - **SageMaker Neo**: トレーニングしたモデルを特定のハードウェア向けに最適化し、クラウドやエ
  - **SageMaker Clarify と SHAP**: 協力ゲーム理論に基づく **SHAP (Shapley Additive Explanations)** のスケーラブルな実装を提供し、モデルの透明性（解釈可能
  - **SageMaker JumpStart**: 事前学習済みのモデルや、すぐ使えるソリューションテンプレート、ノ
- **自動モデルチューニング (AMT) のアルゴリズム特性**
  - **グリッド検索**: 指定範囲の組み合わせを網羅的に試す。
  - **ランダム検索**: 範囲内からランダムな値を選択する。
  - **ベイズ最適化**: 過去の実行結果（回帰モデル）を使って次に試すべき「最も見込みのある」ハイパ
  - **Hyperband**: パフォーマンスの低い学習エポックを早期に自動停止させることで、計算時間とコ
- **学習時間短縮のベストプラクティス (転移学習)**
  - 大規模なネットワークをゼロから学習すると膨大な時間がかかるため、事前学習済みのモデルを
    転移学習 (Transfer Learning) が有効。
  - **SageMaker StudioによるエンドツーエンドのML管理**
  - 単純なノートブックインスタンスとは異なり、AWS管理サーバと**EFS共有ストレージ**を備えた、
  - **ML Lineage Tracking**: データの準備からデプロイまでの系統・来歴を追跡し、ステップの再現
  - **SageMaker Model Registry**: 再現性と監査のためにモデルバージョンを管理。モデルのカタロ

タスク 2.2「モデルのパフォーマンスを評価する」の主な学び

- **モデル評価のためのメトリクス**
  - モデルを用いた予測の正確性を判断するため、問題のタイプごとに適切なメトリクスを選択・理解
  - **分類問題**: 正解率(Accuracy)、適合率(Precision)、再現率(Recall)、偽陽性率(FPR)、特異度(Specific
    スコア、ROC 曲線下面積 (AUC) など。
  - **回帰問題**: 平均二乗誤差(MSE)、二乗平均平方根誤差(RMSE)、平均絶対誤差率(MAPE)
    など。
- **モデル品質のベースライン作成**
  - 学習後に、トレーニングデータを「ベースラインデータセット」としてS3に保存しベースラインシ
- **Amazon SageMaker Model Monitor による品質モニタリング**
  - 時間経過に伴うモデルの精度劣化（品質ドリフト）を監視する。
  - 推論環境（リアルタイムまたはバッチ）でキャプチャされた予測対象データと、後からS3上に保存
    (Ground Truth)を比較し、モデルが当初のベースラインから外れていないか（例:
    回帰ならMSEの悪化など）を評価する。
- **インサイトの取得と学習時のデバッグ (Troubleshooting)**
  - **SageMaker Clarify**: 様々な公平性の概念を定量化し、トレーニング後モデルのバイアスを評価す
  - **SageMaker Debugger**: 学習中にコールバックフックでテンソルを出力・保存。過剰適合、過少

  - **SageMaker with TensorBoard**: トレーニングの経過をダッシュボードで可視化する。
- **モデルの本番デプロイ前評価 (シャドウテスト)**
  - 予測トラフィックをコピーして新しいモデルに流し、現在の本番モデル（インフラ）とパフォーマ
- **モデルの適合 (Model Fit) 改善アプローチ**
  - **過少適合 (Underfitting)**の改善: モデルが学習データをうまく表現できていないため、特徴量を追
  - **過剰適合 (Overfitting)**の改善: モデルが学習データに過剰に最適化（未知のデータに弱い状態）
  - 学習モデルに十分なデータが与えられていない場合、学習・テスト共に精度が落ちるため「単に学

分野3: MLソリューションのデプロイとオーケストレーション

タスク 3.1「既存の要件に基づいてデプロイインフラストラクチャを選択する」の主な学び

- **MLフレームワークと推論**
  - 開発効率化のため、scikit-learn, TensorFlow, PyTorch等のフレームワークと、その上位ライブラリ
    Gluon等）を活用する。
- **要件に応じた推論オプションの選択**（非常に重要な頻出ポイント）
  - **リアルタイム推論**: 常に即座の応答が必要な場合。常時起動のためコストは最も高い。
  - **サーバーレス推論**: アクセスにアイドル期間（空き時間）があり、コールドスタートの遅延を許容
  - **非同期推論 (Asynchronous Inference)**: ペイロードサイズが大きく（最大1GB）、数分〜最大1
  - **バッチ変換 (Batch Transform)**: S3に置かれた大容量データセット全体から一括で予測を得る場
- **SageMaker Inference Recommender**: 負荷テストとチューニングを自動化し、コストとパフォー
  - **SageMaker Neo**: 稼働先のハードウェア（クラウド/エッジ）に合わせてモデルを最適化コンパイ
- **エンドポイントでの A/B テスト運用**
  - 複数のモデルバリアントを比較評価する際、API Gateway等で別々のエンドポイントを複数公開す
  - **単一のSageMakerエンドポイント**の背後に複数のモデルをデプロイし、トラフィックのルーティ
- **MLコンピューティングとスケーリング戦略**
  - コアを最大限に活用するために、場合によって**ミニバッチサイズを増やす**アプローチをとる。
  - **垂直スケーリング (Scale-up)**: インスタンス自体の計算性能を上げる。
  - **水平スケーリング (Scale-out)**: マシンの台数を増やして並列分散処理する。ただし、フレームワ
  - **AWS Inferentia**: 機械学習の推論に特化したAWS独自の高性能・低コストカスタムチップ。推論の
- **Amazon EMR と Spark を活用した分散処理**
  - **EMR Studio (マネージドJupyter Notebook)** を用いてクラスタ上のコードを管理。
  - **Spark MLlib**: EMRに組み込まれたライブラリで、EMRクラスタ内で直接モデルの学習や推論を行
  - **SageMaker Spark ライブラリ**: SageMakerの組み込みアルゴリズム（XGBoost,
    PCAなど）を利用するためのSDK。MLlibの代わりに用いることで、学習や推論環境をSageMake
- **AWS Batch によるジョブ管理**
  - コンピューティング環境、ジョブキュー、ジョブ定義によって構成されるフルマネージドなバッチ
  - コンピューティング環境として **Amazon EC2**（GPUやvCPUの細かな設定が可能）
    や **AWS Fargate**（最大vCPUの指定によりサーバー管理不要）
    を選択でき、いずれもスポットインスタンスを活用してコストを最適化できる。
- **推論パイプラインとエンドポイントの集約**
  - 使用するフレームワークが異なり複数のコンテナが必要な場合でも、1つのエンドポイントの背後
  - **推論パイプライン**: 前処理 → 推論 → 後処理 のロジックを持つ複数のコンテナを、同じインスタン
    低レイテンシー になる。
- **SageMaker MLOps によるCI/CD自動化**
  - **SageMaker Model Registry** を使用し、学習済みモデルのバージョンや承認ステータスを管理。

  - **デプロイガードレール と トラフィックシフト**: 本番で現在のモデルから新しいモデルに切り替える
- **推論パフォーマンスの最適化（再掲・重要）**
  - **AWS Inferentia**: AI、深層学習（画像・NLP等）、生成AIの推論ワークロードをコスト効率良く
  - **SageMaker Neo**: コンパイルを実施することで、モデルの精度を損なうことなく基盤環境やエッ

タスク 3.2「要件に基づいてインフラストラクチャを作成し、スクリプト化する」の主な学び

- **SageMaker エンドポイントのオートスケーリング**
  - 予測不可能な負荷状況においてもモデルを確実に処理させる（スケールさせる）ための第一選択は
  - **ターゲット追跡**: CloudWatchメトリクス（CPU使用率など）の目標値を指定し、それに追従する
  - **ステップスケーリング**: CloudWatchアラームの設定値を超えたタイミングで、段階を追ってスケー
  - **スケジュールスケーリング**: 予測可能なスパイク需要に合わせて、日時を指定して増減させる。
- **エンドポイントのスケールアップ／ダウン**
  - クラスターの台数を増やす（スケールアウト）のではなく、1台の性能を上げる（スケールアップ）
  - 例：大量リクエストを1台で捌くため、Gタイプや**AWS Inferentia**などのアクセラレーターインスタンスに変更する。
  - この場合、既存のエンドポイントに新しい**本番バリアント (Production Variant)** を作成設定し、徐々にトラフィックをシフトさせる運用を行う。
  - ※注意点: Inferentia は性能向上には寄与するが、トラフィック変動への「自動スケール」の解決策
- **コストと管理の最適化オプション**
  - **マルチモデルエンドポイント**: 使用頻度の低いモデルなど、大量のモデルを単一のエンドポイント
  - **サーバーレス推論**: トラフィックが予測できない、あるいはアイドル時間が長い場合に利用。トラフ
  - **マネージドスポットインスタンス**: 中断が許容されるモデルのトレーニングジョブに適用すること
- **Infrastructure as Code (IaC) と監視のベストプラクティス**
  - 環境構築と再利用性を高めるツールとして、**CloudFormation** (テンプレート)、**AWS CDK**（使い慣れたプログラム言語)、**AWS SAM**(サーバーレス環境向け)を選択・活用し、CI/CDに組み込む。
  - エンドポイントの正常性を監視するため、CloudWatchダッシュボードを構築し、モデルレイテン CPU使用率、インスタンスごとの呼び出し数などを継続的にトラッキングする。
- **セキュアなネットワーク通信 (VPC と エンドポイント)**
  - インターネットを経由した通信を避けるため、**VPC インターフェイスエンドポイント (AWS PrivateLink)** を設定する。
  - これにより、VPC内のアプリケーションから、SageMaker API、SageMaker Runtime、SageMaker Studioへの通信が安全な**AWSネットワーク内（プライベ**

タスク 3.3「自動オーケストレーションツールを使用して、CI/CDパイプラインを設定する」の主な学び

- **MLOps の基本原則**
  - バージョン管理、自動化、継続的テスト、そしてモデルガバナンスを標準化し、MLライフサイク
- **対象ごとの『バージョン管理』リポジトリの使い分け**（試験で頻出）
  - **ソースコード / IaC のバージョン管理**: Gitベースのサードパーティーリポジトリ等。ここにコミッ
  - **特徴量のバージョン管理**: **Amazon SageMaker Feature Store**。データから抽出した特徴量をバージョン管理・組織内で参照/再利用できるようにする。
  - **学習済みモデルのバージョン管理**: **Amazon SageMaker Model Registry**。モデルアーティファクト、関連メタデータ、さらに本番環境への「承認ステータス」を
- **様々なデプロイ戦略の使い分け**

  - **ブルー/グリーンデプロイ**: 既存環境（ブルー）と新環境（グリーン）を並行して稼働し、安全かつ
  - **サーバーレス環境のデプロイ (Lambda等)**: 新しいバージョンを発行し、別プロセスで直接呼び出
  - **AWS CodeDeploy の活用**: 線形やCanaryデプロイをサポート。また、トラフィック前後でフッ
- **オーケストレーションツールの選択と統合**
  - ワークフローを繋ぎ合わせるオーケストレーションとして以下のツールを使い分け、AWS CodePipeline等と連携してCI/CDを完成させる。
  - **Amazon SageMaker Pipelines**: データ準備からデプロイまでのプロセスを完全に自動化・管理
  - **SageMaker Operators for Kubernetes**: Kubernetesベース（EKS等）の環境から `kubectl` 等を用いてSageMakerのジョブを直接作成・管理する。
  - **SageMaker Notebook Jobs**: Jupyterで実装したコードそのものをジョブ化し、スケジュール実
  - **AWS Step Functions / MWAA**: AWSの他サービスと連携した巨大なワークフローの中で、Sage
- **CI/CD の実装とテストの自動化**
  - GitFlowに基づき、コードのビルドとテスト（単体テスト等）を行ってアーティファクトを作成する**CI**）。
  - そのアーティファクトをステージング・本番へ自動デプロイし、自動テストを組み込むことでスピ**CD**）。
  - AWS SAM, CodePipeline, CloudFormation等を組み合わせてパイプラインを構成する。大規模な負
- **CI/CD を活用したMLパイプラインの一般的な実装順序とコンポーネント**
  1. 初期のモデル開発とパフォーマンス評価(Notebook等)。
  2. トレーニング・プロビジョニング用のPythonスクリプト準備(コールバック関数の実装等)。
  3. **バッチ推論のモード選択**:
     - **処理ジョブ (Processing Job)**: SageMakerコンテナ内で汎用的なPythonファイルとして実行
     - **バッチ変換ジョブ (Batch Transform Job)**: エンドポイントを拡張し、専用コールバックを
  4. **パイプラインの定義**(Step Functions, MWAA等): ステップ間に入出力の「プレースホルダー」を定
  5. CloudFormation で基盤インフラストラクチャをデプロイ。
  6. **自動化トリガー**: **Amazon EventBridge** がコード、データ、またはモデルの更新イベントを検出